F20/21RO - Intelligent Robotics - Coursework 1

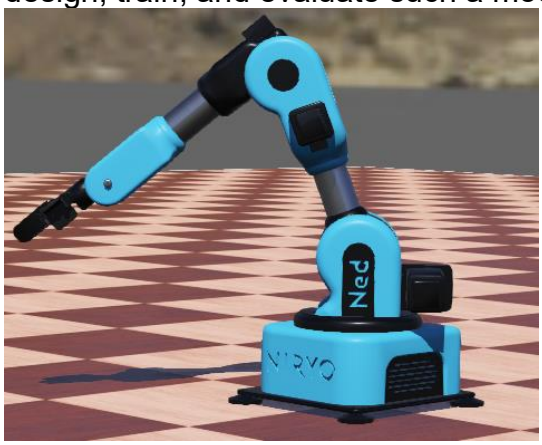
This assessment is worth **30% of your overall course mark**.

Deadline: Friday, 30 October 2026 (Week 7)

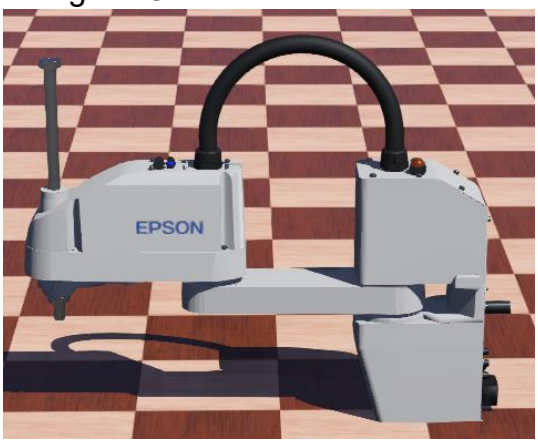
- **Dubai students:** 11:59 PM (Dubai time)
- **Edinburgh students:** 3:30 PM (UK time)

Overview

This assessment is designed to deepen your understanding of kinematics, inverse kinematics, robot simulators, and machine learning by guiding you through the development of an Artificial Neural Network-based approach to controlling two different robot arms in the Webots simulator. You will employ what we learned in the course and combine this with basic machine learning techniques to train a model that can calculate the correct joint values for a robotic arm given a target x, y, z position. Unlike analytical inverse kinematics, a learned model approximates the mapping from end-effector pose to joint values using training data generated from forward kinematics. Your task is to design, train, and evaluate such a model using TWO robot arms in Webots:



The Niryo Ned



The Epson SCARA

The emphasis is on engineering methodology, experimentation, and critical analysis rather than obtaining the lowest possible error. To this end, we encourage the use of Generative AI for the mundane task of producing code in this coursework but require you to follow a conscious and considerate approach to doing so akin to modern practice in the industry (see Important Information). This will require you to understand the code you use for your project and not just blindly trust the output of GenAI.

Support Contacts:

If you encounter any issues within your group or have questions about this coursework that cannot be addressed in one of the lab sessions:

For Edinburgh Campus,
please contact Christian Dondrup at c.dondrup@hw.ac.uk

For Dubai Campus,
please contact Talal Shaikh at t.a.g.shaikh@hw.ac.uk

Important Information

Please read through these other important points before you begin:

- **You do not need to wait until we have covered all the related topics in the lectures to start working on your coursework**.
- We endeavor to give you feedback 15 working days from the submission deadline or your submission (whichever is latest).
- This coursework follows the standard Heriot-Watt rules on:
    - Academic misconduct: https://www.hw.ac.uk/about/professional-services/registry-services/academic-registry/professionalism-and-academic-integrity/academic-misconduct

- Late submission: https://www.hw.ac.uk/document-library/professional-services/registry-services/academic-registry/ltbp-submission-coursework.pdf
- Mitigating Circumstances: https://www.hw.ac.uk/students/manage-your-studies/examinations-and-assessment/mitigating-circumstances

- Peer marking and individual marks - This coursework is completed as a group, but individual students within a group may receive different final marks. The code, poster, video, and AI reflective statement are assessed as group submissions and are used to determine an initial group coursework mark. Individual contribution is not assessed through these group submissions. Every student must also complete an individual peer-marking form through Canvas. This is the only place where you should describe your individual contribution to the project and assess the contributions of the other members of your group. The peer-marking results will be used to adjust the initial group mark to produce individual coursework marks. As a result, students within the same group may receive different final marks.
- The use of AI in the coursework is permitted for some parts but not for its entirety. Please see a list of permitted and not permitted applications of AI below. If something you would like to use generative AI for is not covered by this list, please contact the course team for clarification. If the teaching team has a strong suspicion of a group of students having used generative AI for parts of the coursework which are included in the list of not permitted applications, they will be investigated for plagiarism.
  - **Permitted (examples):** generating or debugging code, explaining errors, suggesting network layout and hyperparameters, generating plotting scripts for graphs, help with mathematical concepts, grammar and spelling, translation, generation of slide layouts, visual templates, or formatting suggestions for the video, generation of poster templates.
  - **Not permitted:** generating final poster or AI statement text, reflective analysis of evaluation results, discussion of lessons learned, discussion of design choices made during implementation, spoken presentation scripts for the video, generation of final video, generation of voiceover for the video, generation of the final technical slide text, final explanations of results, or final discussion/conclusion text, generation of evaluation data, generating the final values in the training/validation/test datasets.

General Task Description

For this coursework, you will explore a different way of controlling a robot arm compared to the lectures. In the lectures, we used mathematical models to control a robot arm, i.e. specify coordinates as to where we want the TCP to be and then calculate the required joint values. Now, you will set up a machine learning-based approach to do the same.

As we discussed in the lecture, the way to control a robot arm is to move its motors. However, using radians for revolute motors or extensions for prismatic motors is not intuitive to use for a human operator. Hence, we now want to implement a way of defining the target location of our TCP and somehow determining the motor values that we need in order to arrive at that location. In the lectures we have done this using trigonometry but now, we will train an Artificial Neural Network to do this. The resulting trained model will be given the target x, y, z position of the TCP as defined by a potential human operator. It is then the task of the trained model to predict the values of the motors of the arm we need to apply to reach this target position.
To make the task more interesting, you are required to control two different arms with the same trained model. This is not only meant to make this more difficult but also to show you how the network does what it does and what the limitations of such an approach are.

To implement your network and train your model, you will work as a group to achieve a set of tasks that build on each other. You are free to organise and distribute the work within your group. Individual contribution should not be reported within the group deliverables; it will be considered separately through the individual peer-marking process. All group members are expected to make a meaningful contribution to the project.

The tasks below build on each other and need to be accomplished in sequence. First, you will build a forward kinematics model for both of your arms in Task 1. Once you have this model, you will use it to create the dataset for training your network in Task 2. While it would be possible to build an inverse kinematics model for both arms mathematically and use that to build the training set, it would defeat the purpose of this exercise. Imagine you would have a more complex arm with more degrees of freedom. This would make it impossible to create a mathematical model for inverse kinematics, but it is always possible to create a forward kinematics model. Hence, to prepare you for real world applications of this technique, we follow the generally applicable approach of creating a forward kinematics model and then using it to create training data for our network.

Once the network has been trained in Task 3, we use it to control the robot arms and evaluate the quality of our trained model in Task 4. Finally, you will report your findings in Task 5.

Before you start

Read the information below before you start your implementation!

General Information

Download the code from gitlab:
https://gitlab-student.macs.hw.ac.uk/f2021ro_2627/nn-ik/ (if you do not have access to this, please contact the course leader of your campus)

Motors to be used for each arm

Robot	Active joint vector	Units	Fixed joints
Ned	`joint_1`, `joint_2`, `joint_3`	radians	all wrist joints fixed: `joint_4`, `joint_5`, `joint_6`
SCARA	`base_arm_motor`, `arm_motor`, `shaft_linear_motor`	radians, radians, metres	end-effector rotation fixed: `shaft_rotation_motor`

For this coursework, we will focus on position only and will disregard orientation. All names given in the table are the motor names used in Webots, e.g. `base = robot.getDevice("base_arm_motor")`. You can find more information on the robots at the end of this document.

Elbow configuration

For this coursework, you will only use the elbow-up configurations of your arm which means that **the elbow motor values (`joint_3` for Ned and `arm_motor` for SCARA) have to be restricted to positive angles only**. More on this in the description of task 2.

Starter code

The gitlab repository provides you with the Webots world files, robot files, a simple joint-control example, the controller stub for use with the red target sphere as seen in the lecture materials, and a supervisor script for reading the TCP position. You will not be provided with partial or complete FK, dataset generation, artificial neural-network training, or evaluation code so you can implement these as you see fit. However, please pay attention to the library dependencies below and the marking rubric and instructions for code submission.

Development environment

For this coursework, we are using Python 3 and Webots 2023b (https://github.com/cyberbotics/webots/releases#release-R2023b) which is the

version that is supported by the GRID computers. You are free to install this on your own machine as well, but we will not be able to support you with any issues that arise from this private installation. **If you use a different version of Webots, your final solution might not work when the teaching team is trying to run it. If that is the case, you will receive 0 marks for this coding related parts of this coursework** as we do not have time to debug individual solutions. Please make sure that your code runs on the GRID computers before submission. Failure to submit code that can be run by the teaching team may result in a mark of 0 for the coursework because the submitted poster and video cannot be verified.

Library dependencies

As you will not be provided with starter code for FK, network, dataset generation, or evaluation, you are free to implement the solution as you see fit with one caveat. To allow the teaching team to run your code, you are only allowed to use the following python libraries: default python libraries (e.g. math, os, sys, etc.), numpy, scikit-learn, and all the libraries provided by Webots. Other libraries such as tensorflow or hugginface are not allowed. This is meant to allow you to implement something yourself instead of just using third party black box components and for the teaching team to be able to run your code without having to install a myriad of libraries first. **If you do use any third-party libraries that are not allowed, you will receive a 0 mark for this coursework.** Failure to submit code that can be run by the teaching team may result in a mark of 0 for the coursework because the submitted poster and video cannot be verified.
The allowed library restriction applies to the submitted executable coursework code. Additional libraries (e.g. matplotlib) may be used to create poster/video figures. The scripts to create these figures do not need to be submitted and are therefore not affected by any restrictions.

Hardcoded dependencies

Changes to world files might result in hardcoded paths that link to a resource on someone's computer. This will mean that the code cannot be run by the teaching team. **If the code cannot be run, you will receive 0 marks.** Please make sure that the code runs either on two different accounts in the GRID lab or two different private laptops before you submit your solution. Failure to submit code that can be run by the teaching team may result in a mark of 0 for the coursework because the submitted poster and video cannot be verified.

Use of generative AI

Please pay attention to the guidance on the use of generative AI in the Important Information section above.

As you will be required to submit a statement on the use of generative AI, we would like to advise you to keep a log of your AI use. What model did you use and what for, how did you prompt it, how did you check the output for correctness, and what impact did this have on your work? This will help you write the AI statement later. Pay attention to the submission instructions and rubric to guide your AI log creation.

TASKS

TASK 1: Forward Kinematics

TASK 1.A

Implement two python controllers in Webots that are able to control the SCARA and Ned arms respectively. They should accept a set of joint values, command the appropriate Webots motors, wait for the arm to settle, and obtain the measured TCP position from the supervisor. You may reuse the scripts provided in the course repository.

TASK 1.B

Implement two python scripts (not controllers), that are able to calculate the forward kinematics of the SCARA arm and the Ned arm respectively.

As input, this should take the active joint values listed in the table above and calculate the resulting x,y,z position of the TCP. Fixed or ignored joints should not be included in the input vector.

TASK 1.C

Evaluate the precision of your forward kinematics calculations from task 1.B using your Webots controller from task 1.A and the supervisor script provided in the gitlab repository. See Robot Specification in the end for more details on the position of the TCP for both arms and try to minimize the error of your forward kinematics scripts.

TASK 2: Creating a dataset

Using your forward kinematics scripts from TASK 1.B, create datasets for training your artificial neural network.

Your python script to create the dataset needs to do the following:

- Create a random distribution of joint values
- Calculate the resulting x, y, z positions of the TCP for all the generated joint values.
- Make sure that there is only one set of joint values for every x, y, z position (elbow-up vs elbow-down). To achieve this, you have to restrict the elbow joint (`joint_3`) of the Ned to positive values only. The same applies to the SCARA arm joint. You should restrict the sampled range of `arm_motor` to positive values only. Use these restrictions consistently for training, validation, and test sets.

- Normalise the joint values to [-1.0, 1.0] using this formula: $q_{\text{norm}} = 2\frac{q - q_{\text{min}}}{q_{\text{max}} - q_{\text{min}}} - 1$
- Split the resulting dataset into a training and validation set (~80/20)
- Save your training and validation sets in files containing the normalised joint value and x, y, z coordinate pairs.

You will need to do this for both arms, the SCARA and the Ned.

You need to create at least 3 different sized training and validation datasets for each arm where the smallest set should have at least 1000 samples before the 80/20 training/validation split. This means you will create a total of 6 training sets and 6 validation sets.
The datasets should be nested, so that the small dataset is a subset of the medium dataset and the medium dataset is a subset of the large one. Otherwise, differences in performance may be caused by different random samples rather than dataset size.
You will need to justify the sizes you chose in the poster/video.

Finally, create two test sets (one for each arm), which you will use to test your artificial neural network's performance in task 4. These test sets should contain at least 500 different joint value/x, y, z position pairs. These pairs should not be in the training or validation sets.

Hence, as a result of this task you should have the following:

- **3 training sets per arm (6 in total)**: used to fit model weights;
- **3 validation sets per arm (6 in total)**: used to select hyperparameters;
- **1 test set per arm (2 in total)**: used only for the final evaluation in task 4.

TASK 3: Train your artificial neural network.

Implement a python script, that can train an artificial neural network to predict joint values based on desired x, y, z coordinates for both the SCARA and Ned arms.

- Create a network structure that is able to predict joint values given x, y, z coordinates.
    - A single network should work for both arms. This will require several adaptations:
        - Add an input flag for the type of arm. The input for the network might then look like [is_scara, is_ned, x, y, z].

- Output needs to be normalised. The output should be three normalised joint values $q_1, q_2, q_3 \in [-1.0, 1.0]$ (just as in your training data) because the actual joint values for the motors on the arms are different (the revolute motors are in radians, the shaft motor is in metres) and also have different min-max ranges.
- Output values need to then be interpreted according to which robot they apply.
  - q_1: Ned -> `joint_1`, SCARA -> `base_arm_motor`
  - q_2: Ned -> `joint_2`, SCARA -> `arm_motor`
  - q_3: Ned -> `joint_3`, SCARA -> shaft_linear`_motor`
  - You can use the following formula to de-normalise the values based on the min and max values for the corresponding robot. See Robot specifications:

$$q = q_{\min} + \frac{q_{\text{norm}} + 1}{2}(q_{\max} - q_{\min})$$

- Train your network using your 6 different training/validation sets. Do not use the test sets at any stage during training!
  - The input for your network should be the x, y, z positions and a flag to identify the robot arm, and the output should be the normalised joint values.
  - Choose the network's hyperparameters, e.g. number of epochs, learning rate, loss function, etc. You will need to justify your choices in the poster/video.
  - Train 3 different models. For each model size, combine the corresponding Ned and SCARA training data into one training set, and combine the corresponding validation data into one validation set.
    - small Ned data + small SCARA data → **small model**;
    - medium Ned data + medium SCARA data → **medium model**;
    - large Ned data + large SCARA data → **large model**.
  - Using your validation sets: refine your hyperparameters and repeat the training until you reach acceptable results. You will need to explain your process in the poster/video.

As a result, you should have 3 trained models: small, medium, and large.

TASK 4: Evaluation

TASK 4.A

This task does not require the simulated arms.

Using the forward kinematics from 1.B and your **test sets** from 2, evaluate the performance of your 3 (small, medium, large) trained models. Conduct these steps for both arms and all 3 models to obtain results:

1. From your test set, give desired coordinates $(x, y, z)_{true}$ to the model.
2. Obtain predicted joint values from your model.
    a. Revolute motors θ_{pred}
    b. Prismatic motors d_{pred}
3. Apply forward kinematics to those predicted values
4. Obtain the resulting TCP position $(x, y, z)_{pred}$.
5. Compare the predicted values to the true values using the metrics below.

Use the following metrics for your evaluation. Compute the error for every sample in the test set. For each model and each robot, report the mean and standard deviation of the position error. For joint errors, report the mean and standard deviation for each active joint.

- Position error: Euclidean distance between $(x, y, z)_{pred}$ and $(x, y, z)_{true}$.
- Joint errors
    - For every joint of the robot, calculate the error:
        - Revolute motors: use the wrapped angular difference: $\Delta\theta = \operatorname{atan2}\left(\sin(\theta_{\text{true}} - \theta_{\text{pred}}), \cos(\theta_{\text{true}} - \theta_{\text{pred}})\right)$
        - Prismatic motors: $e_d = (d_{\text{true}} - d_{\text{pred}})^2$

This should result in:

- 6 sets of position-error statistics: 3 models × 2 arms;
- 18 sets of joint-error statistics: 3 models × 2 arms × 3 active joints.

For each set, report the mean and standard deviation.

TASK 4.B

This task requires the simulated arms.

Using the simulation environment, we are now running another evaluation to confirm the results of your first evaluation. However, we will do this with a much-

reduced test set as it will require more time. Repeat the steps of 4.A with the following alterations:

- Create 2 new tests sets (one for each arm) by using only the first 100 of the test samples from your original test sets. Use those new test sets to obtain the predicted joint values.
- Use the Webots controller+supervisor to obtain the resulting TCP position.
    - Input the predicted joint values into the controller and read the resulting x, y, z coordinates from the supervisor.
    - Compare these values to the x, y, z values in your test set.

This should result in:

- 6 sets of position-error statistics: 3 models × 2 arms.

For each set, report the mean and standard deviation. The joint error statistics can be ignored for this task.

TASK 5: Reporting

Create the required deliverables for this coursework. These are:

- A poster (A0), showcasing:
    - Your network design including justification for your design choices
    - Details on your datasets and justifications for your choices regarding sizes, distribution, and split (training/validation).
    - Evaluation results as graphs/tables comparing the results of 4.A to 4.B.
- A 10-minute video showing:
    - Your network implementation moving both arms in the Webots simulator
        - You may use the red-target-sphere controller example provided on GitLab, as demonstrated in the lectures.
    - Discussions on:
        - Network design choices
            - Influence of the network layout and hyperparameters on performance
            - Influence of dataset size on performance of your network
        - Comparison of results from 4.A and 4.B and reasons of possible discrepancies between the results

    - Lessons learned from the implementation, i.e. programmatic and conceptual issues during development, critical analysis of your final network and its performance
- A 500-word statement on the use of AI for this project
    - Which models did you use and why?
    - How did you check the answers of the models?
    - What did you use the models for?
    - How did the use of AI help your understanding of the topic?
- In addition to these group deliverables, every student must individually submit the peer-marking form provided on Canvas. The peer-marking form is separate from the group deliverables and is the only submission in which individual contribution to the project should be reported.

Robot specifications

In the lectures, we talk about the general idea of how to calculate the forward and inverse kinematics for these robots. To make the maths easier to understand, we use simplifications, e.g. joints being of 0 size. To be able to calculate values for the TCP of the robots in the Webots simulator, you need to account for all the things that we simplified away.

SCARA

Variable	Webots motor	Meaning
q_1	base_arm_motor	base rotation
q_2	arm_motor	second arm rotation
	~~shaft_rotation_motor~~	~~shaft/tool yaw (ignored in this CW)~~
q_4	shaft_linear_motor	vertical shaft displacement

The TCP for the SCARA arm is the end of the shaft where the manipulator would normally be attached.

Note: For the SCARA arm, we are ignoring one of the motors in the middle of the chain. Hence, the third output value of your network model should map to q_4 in the table above. This is slightly confusing as the naming is inconsistent but that is how the arm is built.

Webots shaft convention:

q4 = 0 shaft fully up
q4 = -0.200 shaft fully down

Offsets

Item	Vector / value (x, y, z)
Top-level ScaraT6 frame to joint 1	((0.060, 0, 0.220))
Joint 1 to joint 2 vector	((0.310, 0.005, -0.004))
Joint 2 to TCP/handSlot vector	((0.2545, -0.005, q_4 - 0.034))

This means that the final TCP position can be calculated as:

$$\mathbf{p} = \begin{bmatrix} 0.060 \\ 0 \\ 0.220 \end{bmatrix} + R_z(q_1) \left(\begin{bmatrix} 0.310 \\ 0.005 \\ -0.004 \end{bmatrix} + R_z(q_2) \begin{bmatrix} 0.2545 \\ -0.005 \\ q_4 - 0.034 \end{bmatrix} \right)$$

Joint limits

Coursework variable	Webots motor	Type	Limit
q_1	base_arm_motor	revolute	-0.73 to +0.73 rad
q_2	arm_motor	revolute	-0.83 to +0.83 rad
q_3 ~~(ignored)~~	~~shaft_rotation_motor~~	~~revolute / tool yaw~~	~~-pi to +pi rad~~
q_4	shaft_linear_motor	linear	-0.2 to 0.0 m

Ned

Variable	Webots motor	Meaning
q_1	joint_1	waist/base yaw
q_2	joint_2	shoulder pitch
q_3	joint_3	elbow pitch

The TCP is at the end of the link from the elbow to the first wrist motor.

Offsets

Item	Symbol	Vector / value
Base frame to joint 1	—	((0, 0, 0.1065))
Joint 1 to joint 2	—	((0, 0, 0.0650))
Base frame to shoulder height	(h)	**0.1715 m**
Shoulder to elbow vector	((A,B))	**((0.2210, 0.0120))**
Elbow to teaching TCP vector	((U,V))	**((0.03185, 0.23000))**

Also, we normally assume that for the waist rotation, 0 degrees points along the positive x-axis. However, for the Ned, it points along the positive y-axis. Using the joint offsets, derive the forward kinematics, similar to the SCARA arm above.

Joint limits

Coursework variable	Webots motor	Type	Limit
q_1	joint_1	revolute	-2.8 to +2.8 rad
q_2	joint_2	revolute	-0.8 to +0.8 rad
q_3	joint_3	revolute	-1.5 to +1.3 rad

Submission instructions

For details on the actual submission, please refer to Canvas. For details on what to submit, please see below.

Peer marking

Every student must submit an individual peer-marking form through Canvas. This must be completed independently rather than on behalf of the group. You will be asked to briefly describe your own contribution, provide a self-assessment of your contribution, and assess the relative contributions of the other members of your group. If you believe that the other members contributed equally, you may leave the peer marks at their default equal values. Otherwise, you should adjust them to reflect your assessment of their relative contributions. Instructions for the use of the peer marking form can be found on Canvas.

Code

Code Submission Guidelines

Your group must submit your source code as a **ZIP file including everything you need to run your system (world files, trained models, etc.)**. The code should be readable, organised, and possible for the marker to inspect and understand. Also pay attention to the development environment, allowed libraries and other pitfalls such as hard coded paths. You are not primarily marked on having the most advanced code, but your code should clearly show how your solution was implemented.

Your code submission should include:

1. **Clear file organisation**
    a. Separate scripts for major tasks where possible, for example:
        i. forward kinematics,
        ii. dataset generation,
        iii. artificial neural network training,
        iv. evaluation,
        v. Webots controllers.
2. **Readable code**
    a. Use meaningful variable and function names.
    b. Avoid unnecessary duplication.
    c. Keep functions reasonably short and focused.

3. **Useful comments**
    a. Comment important parts of the code.
    b. In particular, explain:
        i. forward kinematics calculations,
        ii. dataset generation,
        iii. normalisation and denormalisation,
        iv. error calculations,
        v. robot-specific differences between Ned and SCARA.
4. **Instructions for running**
    a. Include a short README file explaining:
        i. which script to run first,
        ii. how to train the model,
        iii. how to run the evaluation,
        iv. how to run the Webots controller.
5. **Reproducibility**
    a. Make sure file paths are relative.
    b. Include or explain where the datasets and trained models should be stored.
    c. State any random seeds used, if applicable.
6. **AI-assisted or reused code**
    a. If you used AI tools or reused code from another source, make sure this is clear.
    b. You remain responsible for checking, understanding, and integrating all submitted code.

The marker should be able to open your submission, understand the structure of the project, and see how the code connects to the results shown in your poster and video. Poorly organised or uncommented code will lose marks even if the final results appear to work.

Poster

Design and layout

Your poster must be submitted as a **single A0 poster** in PDF format. The poster should be a concise visual summary of your project, **not a written report**. Use diagrams, plots, tables, screenshots, and short explanatory captions wherever possible.

Font and sizes: Use **Arial** or a similar sans-serif font, e.g. Calibri, or Helvetica. Main body text: **28pt**, image and table captions: **22pt**, references: **18pt**.

Recommended layout:

1. **Title bar at the top**
   Include the coursework title, names and H numbers of all group members.
2. **Use 3–4 clear columns**
   Organise the poster so that it can be read from left to right and top to bottom.
3. **Suggested section order**
   a. Introduction / Aim
   b. Method Overview
   c. Dataset and Network Design
   d. Results
   e. Comparison and Discussion
   f. Conclusion
   g. References
4. **Prioritise visuals**
   The poster should contain more figures, tables, plots, screenshots, and diagrams than paragraphs.
5. **Keep text short**
   Use bullet points and short captions. Avoid long blocks of text. Detailed explanations should be given in the video.
6. **Make results easy to compare**
   Use consistent labels, units, and colours/styles across graphs for Ned, SCARA, small/medium/large models, FK-based evaluation, and Webots-based evaluation.
7. **Use readable formatting**
   Use Arial with a minimum size of 28pt for the main body. Captions can be 22pt and references 18pt. Avoid overcrowding the poster.

Main Things the Poster Should Contain

Your poster should include the following (this is a non-exclusive list of examples):

1. **Aim of the project**
2. **System overview**
   a. A diagram showing the full pipeline
3. **Robot information**

   a. A compact table or diagram showing which active joints were used for Ned and SCARA.
   b. Indicate which joints were fixed or ignored.
4. **Dataset summary**
   a. Dataset sizes used for small, medium, and large models.
   b. Training/validation/test split.
   c. Short note on sampling strategy.
   d. Short note on how you avoided multiple IK solutions, where applicable.
5. **Network design**
   a. A simple network diagram.
   b. Input format, including the robot identifier flag.
   c. Output format, including normalised joint values.
   d. Key hyperparameters, such as number of layers, number of neurons, activation function, learning rate, loss function, and epochs.
6. **Evaluation metrics**
   a. Indicate whether results were obtained using analytical FK, Webots, or both.
   b. Present joint-error statistics for Task 4.A and position-error statistics for both Task 4.A and Task 4.B.
7. **Results**
   a. Clear plots or tables comparing:
      i. Ned vs SCARA,
      ii. small vs medium vs large models,
      iii. FK-based evaluation vs Webots-based evaluation.
8. **Discussion / interpretation**
   a. 3–5 short bullet points explaining the most important trends.
   b. Mention major limitations or failure cases.
   c. Comment briefly on why performance may differ between the two arms or between FK and Webots evaluation.
9. **Conclusion**
   a. A short statement summarising what worked, what did not work, and what you learned about artificial neural-network-based inverse kinematics.
10. **References**

- Include any references used.

The poster should be understandable as a standalone visual summary of the project. It should not contain all implementation details; those should be explained in the video and supported by the submitted code.

Video

Video Presentation Instructions

Your group must submit a **video presentation no longer than 10 minutes**. If the video is longer than 10 minutes, only the first 10 minutes will be marked. The final video shall not be sped up to save time as this would make narrations impossible to understand.

*Note: Robot motion can be slow, hence source material without sound or narration such as Webots recordings, may be sped up **before** they are added to the final video to save time.*

The video may be recorded using any software you like, for example PowerPoint, OBS, Zoom, Teams, Webots screen recording, or other screen-recording/editing software. The final submitted file must be either:

- **MP4**, or
- **MKV**.

The video should show and explain your final system. It should not simply repeat the poster but you can use it as a visual aid. Use the video to demonstrate the robots working in Webots, explain your implementation, and discuss your results in more detail. The video should include subtitles for all your narrations.

All group members must participate in the video presentation, and **every group member must speak**.

Suggested Video Structure

You may structure the video however you like, but the following structure is recommended:

Section	Suggested time	Content
1. Introduction and aim	~1 min	Briefly explain the goal of the coursework and the two robot arms used.
2. Method overview	~2 min	Explain the main pipeline: FK, dataset generation, artificial neural network training, prediction of joint values, and evaluation.

3. Webots demonstration	~2–3 min	Show the Ned and SCARA arms being controlled in Webots using your predicted joint values. Demonstrate multiple target positions. You can also include an example of a difficult or failed target and explain what happened.
4. Results and evaluation	~2 min	Explain your main results, including position error, joint error, and comparison between small, medium, and large models.
5. Discussion and conclusion	~1–2 min	Discuss what worked, what did not work, limitations, failure cases, and what you learned.

The total video must not exceed **10 minutes**.

Visual Aids

Your video may include (amongst other things):

- Webots screen recordings,
- slides,
- your poster,
- diagrams,
- plots,
- tables,
- screenshots,
- annotated examples,
- short clips showing successful and unsuccessful robot movements.

Visual aids should support the spoken explanation. They should be readable and directly relevant to what is being discussed.

Main Things the Video Should Contain

Your video should include the following:

1. **Project aim**
2. **Brief system overview**
3. **Forward kinematics and dataset generation**
4. **Network design**

5. **Webots demonstration**
6. **Results**
7. **Critical discussion**

The strongest videos will clearly show the system working in Webots, explain how the implementation works, and critically discuss both successful and unsuccessful results.

AI statement

Your group must submit a **500-word AI reflective statement**. You can choose any layout or font. Please keep the size at 11 or higher to increase readability.

The purpose of this statement is to explain how your group used generative AI during the coursework and to reflect on whether it helped your understanding of the topic. You are not marked on whether you used AI a lot, a little, or not at all. You are marked on whether your statement is **honest, specific, reflective, shows responsible use of AI, or explaines why you did not use it**.

Generative AI may include tools such as ChatGPT, Claude, Gemini, Copilot, or similar commercial or open-source systems. The teaching team does not endorse specific models or providers. You as a group are free to choose.

For the creation of this statement, you may use AI for grammar, spelling, or translation support, but the content, reflection, examples, and conclusions of the AI statement must be written by the group.

AI Statement Layout Instructions

Your AI statement should be submitted as a **separate PDF document**.

It should be **500 words (+/- 10%)** using any layout or font at size 11, and should use the following structure:

1. **Title**
    a. Include the names and H numbers of all group members.
    b. Use the title: *AI Reflective Statement*.
2. **Short introduction**
    a. State which AI tools your group used.
3. **Use of AI**
    a. Briefly explain what you used AI for.

4. **Prompting approach**
    a. Describe how you prompted the AI tools.
    b. Mention whether you used broad questions, step-by-step prompts, debugging prompts, requests for explanations, or prompts asking AI to compare alternatives.
    c. Include 2–3 short example prompts or summaries of prompts as an appendix. Does not count towards 500-word limit.
5. **Checking and verification**
    a. Explain how your group checked whether AI-generated suggestions were correct.
6. **Reflection on learning**
    a. Explain whether AI helped your group understand kinematics, inverse kinematics, artificial neural networks, Webots, debugging, or evaluation.
    b. Mention any cases where AI gave incorrect, incomplete, or misleading answers and how your group dealt with this.
7. **Final statement**
    a. Briefly summarise whether your group thinks AI was useful for this coursework and what you learned about using it responsibly.
8. **Appendix**
    a. Any prompts you would like to include as examples.

Main Things the AI Statement Should Contain

Your AI statement should include:

1. **AI tools used**
    a. Name the models or tools used.
    b. Briefly explain why your group chose them.
2. **Tasks AI was used for**
    a. Explain what parts of the coursework AI supported.
    b. Be specific rather than writing “we used AI for help”.
3. **Prompting style**
    a. Describe how your group asked questions.
    b. Include a few short example prompts or prompt summaries in the appendix.
4. **Checking of AI outputs**
    a. Explain how you verified code, mathematical explanations, dataset generation, plots, or debugging advice.
    b. Mention any AI outputs you rejected or corrected.

5. **Impact on understanding**
    a. Reflect on whether AI improved your understanding of any of the topics
6. **Responsible use**
    a. Make clear that the group remained responsible for the final implementation, results, poster, video, and conclusions.

The AI reflective statement should not be a list of tools only. It should show how your group used AI critically, how you checked its outputs, and whether it helped you understand the coursework topic. Honest discussion of mistakes, limitations, or incorrect AI suggestions is encouraged.

Marking criteria

As this coursework allows the use of AI for the generation of code, you are not marked simply on achieving the lowest numerical error. However, your implementation must run, produce meaningful results, and support the analysis presented in your poster and video. Marks are awarded for the quality of your implementation, evaluation, explanation, and critical interpretation of the results.

There will also be some marks for code quality, i.e. comments and readability, and a general statement and discussion about your use of AI for this coursework.

Failure to submit code that can be run by the teaching team may result in a mark of 0 for the entire coursework because the submitted poster and video cannot be verified.

The marking process takes place in two stages. First, the group submissions are marked using the criteria and rubrics below to produce an initial group coursework mark. Individual contribution is not considered when marking these group submissions. The initial group mark is based on the following components:

- Code (10%)
- Poster (30%)
- Video (45%)
- AI statement (15%)

The initial group coursework mark is then adjusted for individual students using the peer-marking assessments submitted by the members of the group. Students within the same group may consequently receive different final coursework marks.

Marking rubrics

The rubrics below are used to assess the quality of the group submissions and determine the initial group mark. They do not assess individual contribution to the project. Individual contribution is considered separately through peer marking after the group submissions have been marked.

Code (10%)

Code Quality, Readability and Reproducibility	10%
A (70–100%)	The submitted code is very easy to read, well organised, and logically separated into appropriate components such as forward kinematics, dataset generation, training, evaluation, and Webots control. Naming is clear and consistent, comments explain important calculations, assumptions, design choices, normalisation, and error metrics, and the submission includes clear instructions so the marker can reproduce the main results with minimal effort. The code shows strong evidence of careful human review and understanding, including appropriate integration of any reused or AI-assisted code.
B (60–69%)	The code is mostly readable, organised, and reproducible. The main components are separated sensibly, naming is generally clear, and comments explain most important parts of the implementation. Instructions are sufficient for the marker to run the code with only minor assumptions or corrections. There is good evidence that the group has reviewed and understood the submitted code.
C (50–59%)	The code is understandable with some effort. The main scripts are present, but organisation, naming, commenting, or instructions may be inconsistent or incomplete. Some important assumptions, mathematical calculations, or execution steps may not be clearly documented. The submission provides some evidence of understanding, but parts of the code may appear only loosely integrated.
D (40–49%)	The code is difficult to follow or reproduce. File organisation is unclear, naming is inconsistent, comments are sparse or superficial, and important

	assumptions or instructions are missing. The marker would need to make substantial effort to understand or run the code. There is limited evidence that the group has fully reviewed or understood all submitted components.
E/F (<40%)	The code is disorganised, largely unreadable, undocumented, or not meaningfully reproducible. It is unclear how the submitted files relate to the coursework tasks, or the code appears pasted together without meaningful human review. Instructions are missing or unusable, and the marker cannot reasonably inspect, understand, or run the submitted implementation.

Poster (30%)

Method and System Overview	6%
A (70–100%)	Clear visual overview of the full pipeline: FK, dataset generation, artificial neural network training, evaluation, and Webots demonstration. Includes both Ned and SCARA and shows how the shared model handles both robots.
B (60–69%)	Good overview of the main pipeline and both robots. Most key steps are shown, but one or two details may be less clear.
C (50–59%)	Basic pipeline is shown, but the connection between the different stages is not always clear. Both robots are mentioned, but not equally well represented.
D (40–49%)	Limited or confusing overview. Important stages of the method are missing or poorly connected.
E/F (<40%)	No meaningful overview of the system or workflow. The poster does not make clear what was implemented.

Dataset and Network Design	6%
A (70–100%)	Concisely shows dataset sizes, train/validation/test split, sampling approach, nesting, and the network input/output format. Includes a clear diagram or compact table showing input and normalised joint-value outputs.
B (60–69%)	Shows the main dataset and network design choices clearly. Minor details, such as nesting or normalisation, may be less explicit.
C (50–59%)	Gives basic dataset and network information, but some required details are missing or unclear.
D (40–49%)	Dataset or network design is only partially described. The reader cannot easily tell how the model was trained.
E/F (<40%)	Dataset and network design are absent, incorrect, or too vague to assess.

Results and Visual Evidence	8%
A (70–100%)	Presents clear plots/tables for both robots and all three model sizes. Includes error summaries using mean and standard deviation. Visuals are labelled, readable, and easy to compare.
B (60–69%)	Presents the main results clearly for both robots. Most required metrics and comparisons are included, with only minor omissions or clarity issues.
C (50–59%)	Some results are shown, but comparisons are incomplete, hard to read, or missing some models, robots, or metrics.
D (40–49%)	Results are limited, poorly labelled, or difficult to interpret. Important comparisons are missing.
E/F (<40%)	Little or no meaningful result data is shown. Results are missing, unrelated, or impossible to understand.

Comparison and Critical Interpretation	6%
A (70–100%)	Provides concise but insightful interpretation of the results using short captions, annotations, or bullet points. Clearly compares Ned vs SCARA, small/medium/large datasets, and FK-based vs Webots-based evaluation. Identifies key limitations.
B (60–69%)	Provides a clear comparison and identifies the main trends and limitations. Interpretation is mostly supported by the presented results.
C (50–59%)	Includes some interpretation, but it is mostly descriptive. Trends are identified but not explained in much depth.
D (40–49%)	Very limited interpretation. The poster shows results but gives little explanation of what they mean.
E/F (<40%)	No meaningful comparison or interpretation is provided. Conclusions are unsupported or missing.

Poster Design, Clarity, and References	4%
A (70–100%)	Poster is professional, readable, and well structured. Uses space effectively with appropriate balance between text and visuals. Figures/tables are labelled. References are included clearly.
B (60–69%)	Poster is clear and readable with a sensible layout. Visuals mostly support the content. References are included.
C (50–59%)	Poster is understandable but may be cluttered, text-heavy, or visually inconsistent. References may be present but limited.
D (40–49%)	Poster is difficult to follow due to poor layout, excessive text, missing labels, or weak structure. References may be incomplete.
E/F (<40%)	Poster is very difficult to read or does not communicate the project effectively. References statement is missing.

Video (45%)

Webots Demonstration and System Functionality	10%
A (70–100%)	The video clearly demonstrates the artificial neural-network-based controller operating in Webots for both the Ned and SCARA arms. The demonstration shows the robots attempting multiple target positions and makes clear how the network predictions are converted into robot joint commands. Successful cases and limitations or failures are both shown honestly.
B (60–69%)	The video demonstrates both robots in Webots and shows the main functionality clearly. The explanation of how predictions are used to move the robots is mostly clear, although some details or limitations may be less well shown.
C (50–59%)	The video includes a basic Webots demonstration, but it may be limited, difficult to see, or not fully connected to

	the artificial neural network implementation. Both robots may be shown, but not equally well.
D (40–49%)	The Webots demonstration is weak, incomplete, or difficult to follow. It is unclear whether the submitted system works reliably, or one robot may be largely missing from the demonstration.
E/F (<40%)	There is no meaningful Webots demonstration, or the video does not show the coursework system operating.

Technical Explanation of the Implementation	**12%**
A (70–100%)	The group gives a clear and technically accurate explanation of the implementation, including forward kinematics, dataset generation, network input/output format, robot identifier input, normalisation/denormalisation, and evaluation workflow. The explanation shows strong understanding of how the different parts of the system connect.
B (60–69%)	The implementation is explained well. Most major components are described correctly, with only minor omissions or lack of detail. The connection between the kinematics, artificial neural network and Webots controller is mostly clear.
C (50–59%)	The video gives a satisfactory explanation of the implementation, but some important details are missing, vague, or only described superficially.
D (40–49%)	The technical explanation is limited or unclear. Important parts of the implementation are missing, inaccurate, or difficult to understand.
E/F (<40%)	The video does not provide a meaningful technical explanation of the implementation, or the explanation shows fundamental misunderstanding.

Results, Evaluation and Interpretation	**12%**

A (70–100%)	The video clearly explains the main evaluation results, including performance on both robots, comparison of small/medium/large models, error metrics, and differences between FK-based and Webots-based evaluation. The group gives thoughtful reasons for observed trends and discrepancies.
B (60–69%)	The main results are explained clearly and linked to the coursework metrics. The discussion identifies important trends and gives reasonable explanations, although some analysis may lack depth.
C (50–59%)	The video presents some results, but the explanation is mostly descriptive. Some required comparisons or metrics may be missing or not fully interpreted.
D (40–49%)	Results are mentioned but poorly explained. The video provides little meaningful interpretation of the evaluation or comparison between robots/models.
E/F (<40%)	The video does not meaningfully present or interpret the evaluation results.

Use of Visual Aids, Structure and Clarity	**5%**
A (70–100%)	The video is well structured, easy to follow, and makes excellent use of visual aids such as slides, diagrams, plots, code snippets, Webots screen recordings, or annotated results. Visuals and subtitles are readable and support the spoken explanation. Timing is well managed.
B (60–69%)	The video is clearly structured and uses appropriate visual aids and subtitles. Most visuals are readable and relevant, with only minor issues in flow, pacing, or clarity.
C (50–59%)	The video is understandable but may have some issues with structure, pacing, visual clarity, coherence of subtitles, or over-reliance on text-heavy slides.
D (40–49%)	The video is difficult to follow. Visual aids and subtitles are rare or missing entirely, hard to read, poorly

	explained, or not well connected to the spoken presentation.
E/F (<40%)	The video is poorly structured, very difficult to understand, or lacks useful visual support and subtitles.

Presentation Participation and Quality	**6%**
A (70–100%)	All group members participate meaningfully in the video presentation and speak clearly. The presentation is well rehearsed, professional, coherent, and balanced. The group demonstrates shared understanding of the project.
B (60–69%)	All group members participate in the presentation. Delivery is generally clear and well prepared, with only minor imbalance or presentation issues.
C (50–59%)	All or most group members participate, but presentation time or delivery may be uneven. The presentation is understandable, although some members may provide less clear explanations than others.
D (40–49%)	Participation in the presentation is substantially unbalanced or the delivery is poorly prepared. One or more members may have very limited participation, and the presentation is difficult to follow in places.
E/F (<40%)	The presentation is unprofessional or incoherent, or the requirement for all group members to participate and speak has not been met.

AI statement (15%)

You are not marked on whether you used AI a lot, a little, or not at all. You are marked on whether your statement is honest, specific, reflective, and demonstrates responsible engineering practice.

AI Reflective Statement	**15%**
A (70–100%)	The statement gives a clear, honest, and specific account of AI use. It identifies which AI models/tools were used and explains why they were chosen. It

	describes the group's prompting style with useful examples or summaries of prompts, clearly explains what AI was used for, and gives concrete examples of how outputs were checked, corrected, or rejected. The statement reflects thoughtfully on whether AI helped the group understand kinematics, inverse kinematics, artificial neural networks, debugging, evaluation, or Webots. It shows responsible and critical use of AI rather than simple reliance on generated answers.
B (60–69%)	The statement clearly explains the main AI tools used, why they were used, and what they were used for. It describes the prompting approach and gives some indication of how outputs were checked. The reflection on understanding is sensible, although it may not be very detailed or strongly linked to specific coursework examples.
C (50–59%)	The statement gives a basic description of AI use. It names the tools used and describes some tasks AI supported, but the explanation of prompting, checking, or impact on understanding is limited. The statement is mostly descriptive rather than reflective.
D (40–49%)	The statement is vague or superficial. It may mention AI tools but gives little detail about why they were chosen, how they were prompted, what they were used for, or how outputs were checked. Reflection on learning and understanding is weak or generic.
E/F (<40%)	The statement is missing, extremely brief, unclear, or does not meaningfully address the required points. It gives little or no evidence of responsible AI use, checking of outputs, or reflection on how AI affected the group's understanding.